

Systemd Unit, Service, and Timer Management Guide

This document provides a comprehensive overview of managing systemd units, services, and timers, based on established best practices and custom implementation patterns.

1. Core Concepts: Units, Services, and Timers

What is a Unit?

In systemd, a **Unit** is any resource that the system knows how to manage. This includes services, timers, sockets, device nodes, and mount points. Every unit is defined by a configuration file (e.g., `.service` or `.timer`).

Services vs. Timers

- **Service (`.service`):** Defines *what* to run. It contains the logic to execute a script or binary.
 - **Timer (`.timer`):** Defines *when* to run. It triggers a corresponding service based on time intervals or calendar events.
-

2. Directory Structure & Precedence (Where files live)

Systemd looks for unit files in several places. The order (precedence) is crucial:

1. `/etc/systemd/system/` (Highest Priority)
 - **Purpose:** This is where you (the administrator) put your custom units.
 - **Behavior:** Files here always override files with the same name in other directories. This is where `systemctl enable` creates symbolic links.
 2. `/run/systemd/system/`
 - **Purpose:** Runtime units created dynamically (e.g., by mounting a drive). These are lost on reboot.
 3. `/lib/systemd/system/` or `/usr/lib/systemd/system/` (Lowest Priority)
 - **Purpose:** System-installed or “Vendor” units.
 - **Warning:** Never edit files here directly! If a package update occurs, your changes will be overwritten. To change these, copy the file to `/etc/systemd/system/` and edit the copy.
-

3. Anatomy of a Service File (.service)

Standard location for custom work: /etc/systemd/system/

```
[Unit]
Description=Human-readable description of the service
After=network.target          # Start only AFTER these units are
active                        #
Wants=network.target          # Weak dependency: try to start these
too                           #

[Service]
ExecStart=/path/to/script.sh # The command to execute
Restart=on-failure            # Automatic restart logic
RestartSec=10                 # Delay before restarting

[Install]
WantedBy=multi-user.target    # Defines which "target" triggers this
on boot
```

Key Directives Explained

- **After=**: Specifies ordering. If Service A has After=Service B, A will wait for B to finish starting. It does *not* create a requirement dependency.
 - **Wants=**: A “weak” dependency. If this unit starts, it will try to start the listed units, but it won’t fail if they don’t start.
 - **WantedBy=**: Used when **enabling** a unit. It creates a symbolic link in the .wants/ directory of the specified target.
-

4. Anatomy of a Timer File (.timer)

A timer typically shares the same name as the service it controls (e.g., backup.timer triggers backup.service).

```
[Unit]
Description=Timer for periodic task

[Timer]
OnCalendar=*:0/15             # Run every 15 minutes
Unit=custom-task.service      # Explicitly specify the service to
trigger                       #

[Install]
WantedBy=timers.target
```

OnCalendar Examples

- *:0/1: Every second.
 - *:0: Every minute.
 - 0:0: Every hour.
 - *-*-* 00:00:00: Daily at midnight.
-

5. Understanding Targets (WantedBy)

Targets are grouping mechanisms (similar to runlevels). * **default.target**: The default target reached on boot (usually a symlink to graphical or multi-user). * **multi-user.target**: A non-graphical system state (server mode). * **graphical.target**: A system with a GUI. * **timers.target**: Used specifically for timers to ensure they start during the timer-initialization phase.

6. Management Lifecycle (The systemctl Tool)

Creating / Making Units

1. Create your script (e.g., in `/usr/local/bin/` or `/etc/systemd/system/CUST-SYSD/`).
2. Ensure it is executable: `chmod +x /path/to/script.sh`.
3. Create the `.service` (and optionally `.timer`) file in `/etc/systemd/system/`.
4. **Critical**: Run `systemctl daemon-reload` every time you modify a unit file.

Enable vs. Start

- **systemctl start [unit]**: Runs the unit **immediately** for the current session.
- **systemctl enable [unit]**: Configures the unit to start **automatically on boot**.

Disable vs. Stop

- **systemctl stop [unit]**: Terminates the running process **now**.
- **systemctl disable [unit]**: Removes the boot-time triggers (removes symlinks).

Deleting / Removing Units

1. `systemctl stop [unit]`
 2. `systemctl disable [unit]`
 3. `rm /etc/systemd/system/[unit]`
 4. `systemctl daemon-reload`
 5. `systemctl reset-failed` (Optional: clears the “failed” state from memory).
-

7. Status and Troubleshooting

- **Check Status**: `systemctl status [unit]`
- **View Logs**: `journalctl -u [unit]`

- **List All Timers:** `systemctl list-timers --all`
-

8. Custom Implementation Patterns

Reflected from the “Systemd Unit and Timer Manager” Script:

- **Isolated Scripts:** Store executable logic in subdirectories like `/etc/systemd/system/CUST-SYSD/` to keep the main unit directory clean.
- **Safety Buffers:** Use `ExecStartPre=/usr/bin/sleep 20` to allow hardware/drivers to settle before a script runs.
- **Naming Conventions:** Prefixing custom units with `CUSTOM-` makes them easy to find and batch-manage using `grep`.